

ProBuild — Socio-Technical Model (iStar 2.0) with Rational Justification

This document specifies the Strategic Dependency (SD) and Strategic Rationale (SR) models that the operational platform fulfils, and — importantly — *justifies* the modelling choices. It follows the iStar 2.0 language (Dalpiaz, Franch and Horkoff, 2016), which builds on the original i* framework for early-phase requirements engineering (Yu, 1997). The `.txt` Pistar sources and rendered diagrams live alongside this file (`SDmodel1/` , `SRmodel1/`).

The model is the contract. Every actor becomes a pool or a capability process; every strategic dependency becomes a call-activity contract or a correlated message; every softgoal shapes a design decision. The traceability table in §4 names, for each i* element, where it surfaces at runtime.

1. Actors and why they exist

i* Actor	Kind	Realised as	Why a separate actor
Customer	Role	<code>Journey_Customer</code> start / user tasks	The initiator; owns the goals the whole system serves
ProBuild (orchestrator)	Agent	<code>Journey_Customer</code> orchestrator	Owens the end-to-end experience; delegates rather than doing
Sales	Agent	<code>Cap_Sales</code>	A reusable capability with its own goal (accurate priced quote)
Warehouse	Agent	<code>Cap_Warehouse</code>	Physical fulfilment + returns; called by <i>two</i> journeys — justifies its own boundary
Loyalty	Agent	<code>Cap_Loyalty</code>	Cross-cutting membership concern invoked from card <i>and</i> buy journeys
Tool Hire	Agent	<code>Cap_Hire</code>	Distinct commercial model (deposits, tiered daily pricing)
FinTrust UK	External agent	<code>Partner_FinTrust</code> pool	Independent organisation; can only be reached by message, never by call
FixPro Ltd	External agent	<code>Partner_FixPro</code> pool	Independent maintenance provider; autonomous, message-integrated
Supplier	External agent	backorder dependency	Out of automation scope but a real dependency for stock

Design choice justified. ProBuild is modelled as an *orchestrator that depends on internal capability agents*, not as one monolithic agent. This is deliberate: it makes each capability independently satisfiable and reusable (Warehouse is depended upon by both Sales-fulfilment and Hire-returns),

and it keeps the two *external* actors strictly at arm's length — they are reached only through message dependencies, never internal calls, reflecting the real organisational boundary.

2. Strategic Dependencies (SD)

- Customer → ProBuild: *smooth purchasing experience* (softgoal), *goods delivered, card issued*
- ProBuild → Sales: *accurate, priced quote* (task), *real-time stock truth* (resource)
- ProBuild → Warehouse: *order fulfilled, return assessed*
- ProBuild → Loyalty: *points & band maintained*
- ProBuild → Tool Hire: *tool booked at fair price*
- ProBuild → FinTrust: *finance decision* (goal, by message), *funds transferred* (resource)
- ProBuild → FixPro: *tools serviced & compliant* (goal, by message), *service report* (resource)
- Sales / Tool Hire → Supplier: *stock replenished* (resource)

3. Strategic Rationale (SR) — goals, tasks, softgoals

Actor	Hard goals	Softgoals (the "-ilities")
ProBuild	Serve customer end-to-end; keep every step auditable	<i>Smooth experience, no dead ends, traceability</i>
Sales	Produce a correct priced quote; handle out-of-stock	<i>Accurate real-time pricing</i>
Warehouse	Fulfil orders; assess returns	<i>Accurate inventory, quality (QC loop)</i>
Loyalty	Issue/renew cards; accrue points; band correctly	<i>Fair, transparent rewards</i>
Tool Hire	Book tools; price by duration	<i>Fair, transparent hire pricing</i>
FinTrust	Assess creditworthiness; settle	<i>Responsible lending, fast decision</i>
FixPro	Service batch; certify (PAT)	<i>Tools safe & compliant, minimise downtime</i>

Softgoal-conflict analysis (why "rational justification" matters)

Real design tension exists between softgoals, and the architecture takes an explicit position:

- **Fast decision vs Responsible lending** (FinTrust). Resolved by a **DMN + human** split: the `finance_assessment` decision table gives an instant recommendation (fast), but a human reviewer confirms it (responsible). Neither softgoal is sacrificed.
- **Fair transparent pricing vs Revenue** (Tool Hire, Loyalty). Resolved by moving pricing and banding into **DMN decision tables** — the rules are inspectable and uniform, so "fair and transparent" is structurally guaranteed rather than asserted.
- **Quality vs Throughput** (Warehouse). Resolved by the **QC re-pick loop**: an order cannot dispatch until QC passes, favouring quality, but the loop is local so throughput is only affected when defects are found.
- **Autonomy vs Coordination** (FinTrust, FixPro). Resolved by **message choreography**: partners run fully autonomously (own pools, own logic) yet coordinate through two correlated messages — loose coupling without losing the end-to-end thread (`journeyId`).

Alternatives considered (and rejected)

- *One ProBuild pool with internal lanes* (the classic approach). Rejected: it couples capabilities that should be independently deployable/testable and blurs the external-partner boundary.
- *Pure event choreography, no orchestrator*. Rejected: it makes the single-customer journey hard to reason about and demo, and scatters the "smooth experience" softgoal with no owner.
- *Hard-coded business rules in Java*. Rejected: it hides the "fair, transparent" softgoals inside code. DMN externalises them as first-class, inspectable decisions.

4. Traceability — i* element → runtime artefact

i* element	Type	Where it surfaces
Smooth purchasing experience	Softgoal	Orchestrator drives the whole journey; one instance, no re-keying
Accurate real-time pricing	Softgoal	<code>sales-check-inventory</code> prices from live INVENTORY
Finance decision	Goal (msg)	<code>Partner_FinTrust</code> started by <code>finance.requested</code> ; <code>finance_assessment</code> DMN
Responsible lending	Softgoal	DMN recommendation + human <code>FT_Review</code> confirmation
Fair, transparent hire pricing	Softgoal	<code>hire_pricing</code> DMN (tier multiplier, inspectable)
Fair, transparent rewards	Softgoal	<code>loyalty_band</code> DMN (band thresholds, inspectable)
Tools safe & compliant	Goal (msg)	<code>Partner_FixPro</code> ; <code>maintenance_triage</code> DMN + PAT step
Order fulfilled	Hard goal	<code>Cap_Warehouse</code> pick → QC → dispatch; ORDERS row DISPATCHED
No dead ends	Softgoal	Every reject path reaches <i>Customer cancelled</i> cleanly
Auditable steps	Softgoal	Every worker writes an H2 row + a log line

Note. The SD/SR diagrams (`SDmodel/` , `SRmodel/`) were produced in the Pistar 2.1.0 tool (<https://www.cin.ufpe.br/~jhcp/pistar/>) from the `.txt` sources; this document is their authoritative specification and justification.

References

- Dalpiaz, F., Franch, X. and Horkoff, J. (2016) *iStar 2.0 Language Guide*. arXiv:1605.07767.
- Yu, E. (1997) 'Towards modelling and reasoning support for early-phase requirements engineering', *Proceedings of RE'97*. Washington, DC: IEEE, pp. 226–235.

(Full list: `docs/references.md`.)